

SECAUDIT 048 — Live Penetration Test of 0.7.527

Test date: 2026-08-15

Target release: SnapSmack 0.7.527D

Test environment: Disposable GRAMOFSMACK installation, isolated from the production network hub

Remediation release: 0.7.528D

Publication status: EMBARGOED until 0.7.528D deployment is independently verified on the tested site

Executive summary

SECAUDIT 048 was SnapSmack's first live penetration test. Earlier reviews read the code and reasoned about what an attacker might reach. This test attacked a running installation as both an unauthenticated visitor and an authenticated content-only editor.

The live test found two critical installation-state failures, one high-severity script-injection flaw, and one medium-severity authorization gap. The critical failures could expose recovery and deployment machinery after installation. The script-injection flaw allowed stored content to escape a JSON-LD data block and execute in a visitor's browser. The authorization gap let an editor reach three system pages that should have been restricted to administrators.

All critical, high, and medium findings were corrected in 0.7.528D. The test also identified four low-severity hardening items. Two were completed in 0.7.528D; two remain scheduled for the next maintenance release and do not provide account takeover, code execution, or data access on their own.

The exercise also verified that many defenses added during SECAUDIT 047 held up under real attack: administrative authentication, sensitive-file blocking, path-traversal protection, upload restrictions, signed download links, destructive-action CSRF tokens, session-cookie protection, login response handling, password-reset rate limiting, and public-registration controls.

Scope and method

The test used a disposable site with no production content and no connection to the SnapSmack network hub. Testing covered two realistic threat positions:

- 1 an unauthenticated internet visitor; and
- 2 an attacker who had obtained the password of a content-only editor.

The second position matters because editor accounts are intentionally allowed to create and manage content but must never gain control of users, site-wide configuration, recovery, updates, secrets, or system maintenance.

Tests included access-control checks, installation-state checks, controlled upload attempts, stored and reflected script-injection probes, path traversal, sensitive-file requests, destructive-action CSRF review, download-token tampering, session behavior, login behavior, password-reset behavior, and editor-role boundary checks.

No production site, production account, private archive, or third-party system was used as a target.

Findings at a glance

#	Severity	Finding	Live proof	Resolution
1	CRITICAL	Installed site could still expose installer recovery/patch functions	Confirmed	Fixed in 0.7.528D
2	CRITICAL	Bootstrap deployer could become available again after installer removal	Confirmed	Fixed in 0.7.528D
3	HIGH	Stored/reflected script injection through JSON-LD output	Confirmed, script executed	Fixed in 0.7.528D
4	MEDIUM	Three system pages were missing from the editor restriction boundary	Confirmed	Fixed in 0.7.528D
5	LOW	HSTS response header absent	Confirmed by headers	Fixed in 0.7.528D
6	LOW	Content-Security-Policy header absent	Confirmed by headers	Initial policy added in 0.7.528D
7	LOW	Editor dashboard exposes unnecessary server diagnostics	Confirmed	Scheduled for next maintenance release
8	LOW	Password-reset request form lacks a CSRF token	Confirmed	Scheduled for next maintenance release

Critical finding 1 — installed-site recovery surface

What happened

SnapSmack correctly recognized that the site was already installed during the normal installation path, but two exceptional installer modes were evaluated before the installed-state lock fully took effect. That left recovery and

schema-patching capabilities reachable without first entering the authenticated administration area.

Why it mattered

Installation and recovery code has unusual power: it can inspect or replace files and alter database structure. Leaving any of that machinery reachable after installation creates an unauthenticated route around the normal admin, two-factor, and recovery controls. The practical consequence was potential server takeover or destructive database changes.

Correction

Once an installed SnapSmack database is detected, `install.php` now refuses all installer actions. Recovery for an existing site is available only through the authenticated Disaster interface. The blank-server recovery path remains available only when no installed site exists.

Critical finding 2 — bootstrap deployer reactivation

What happened

The small bootstrap deployer used an overly narrow test to decide whether a site already existed. Removing one installation file—an otherwise sensible cleanup step—could make the bootstrapper believe deployment was available again even though the application and database configuration were present.

Why it mattered

A deployment bootstrapper can write the application onto the server. If it reactivates on a live installation, an unauthenticated visitor may be able to replace or redeploy files over the existing site.

Correction

`setup.php` now locks immediately when either of the durable installed-site markers exists. It returns a refusal page instead of entering any deployment path. Removing the main installer no longer re-arms the bootstrapper.

High finding — JSON-LD script injection

What happened

Photo titles, captions, ALT text, tags, and the current page URL were serialized into a JSON-LD `<script>` element. The JSON itself was valid, but the output did not hex-escape HTML-significant characters. A stored closing-script sequence

could therefore terminate the data block and introduce executable markup. The test demonstrated both stored and reflected execution on the disposable site.

Why it mattered

An attacker able to submit or edit content could run script in another visitor's browser under the blog's origin. Depending on the victim and page, that could alter content, perform authenticated actions, or expose information available to the browser. The HttpOnly session cookie correctly prevented direct JavaScript access to the session identifier, limiting—but not eliminating—the impact.

Correction

JSON embedded in script elements is now emitted with the full set of PHP JSON hex-escaping flags for tags, ampersands, apostrophes, and quotation marks. The same protection was applied to the inline configuration JSON. The resulting JSON-LD remains valid for search engines while closing the HTML parser boundary.

Medium finding — incomplete editor boundary

What happened

SECAUDIT 047 introduced a central admin-only page boundary, but three system pages were omitted from its registry. A content-only editor could open traffic statistics and reach audit/backfill functions that write settings or image records.

Why it mattered

The role model promises that editors can manage content without controlling the site. Any omitted system page weakens that promise and may provide a path to configuration changes, bulk mutation, or future privilege escalation.

Correction

The audit, backfill, and statistics pages are now part of the central admin-only registry. The existing gate returns a refusal for editor sessions before page logic runs. Administrator-authenticated backup tooling is unaffected.

Low-severity hardening

Completed in 0.7.528D

- **HSTS:** HTTPS responses now instruct supporting browsers to remain on HTTPS for one year, including subdomains.

- **Initial CSP:** responses now prohibit plugins/embedded objects, hostile base URL rewriting, and third-party framing. A stricter source policy for scripts and images remains a separate compatibility project because SnapSmack can display federated media.

Scheduled for the next maintenance release

- **Editor diagnostics:** remove PHP version, server software, filesystem/load, and cron-path details from the content-only editor dashboard. This is unnecessary information disclosure but does not grant access by itself.
- **Reset-request CSRF:** add a token to the password-reset email request. The current endpoint sends only a rate-limited reset message and uses the same response for known and unknown addresses; it does not reset a password without the emailed secret.

These two low findings are disclosed as open maintenance items only after the serious findings are confirmed closed. Neither is being represented as fixed.

Defenses that held up

The following controls were exercised against the running site and resisted the test:

- unauthenticated access to normal admin pages was redirected to sign-in;
- sensitive configuration, repository, environment, migration, and database files were not publicly readable;
- path-traversal and local-file probes did not escape their intended routes;
- executable uploads were rejected;
- a deliberately malformed image/program polyglot was stored as inert image data and did not execute;
- signed download links rejected forged and modified tokens;
- destructive media and post actions carried CSRF tokens;
- the session identifier remained HttpOnly;
- login failures did not expose sensitive detail;
- password-reset requests were rate-limited and did not reveal whether an address belonged to an account;
- public self-registration was disabled.

This list is important: a penetration report should record not only what failed, but which security claims were actually tested and survived.

Limits of this test

Federation was disabled on the disposable site, so live ActivityPub signature forgery and remote actor behavior were outside this pass. The actor/key binding introduced after SECAUDIT 047 was reviewed in code but must receive a separate live federation test when an isolated federated fixture is available.

This was a targeted test, not proof that no other vulnerability exists.

Remediation verification

The 0.7.528D patch set was reviewed after implementation:

- installed-state checks now run before installer recovery, patch, or deployment behavior;
- embedded JSON uses HTML-safe JSON encoding while remaining parseable JSON-LD;
- the three omitted system pages pass through the existing central administrator gate;
- HSTS and the initial CSP are emitted by the shared public response-header path;
- all changed PHP files passed syntax validation;
- all changed source files retained the required SnapSmack EOF markers.

Live closure requires one final deployment verification on the disposable test site. Until that verification succeeds, this report remains embargoed and must not be copied into the public Buzzers directory.

Closure gate for publication

Publish this report only after all of the following are true:

- 1 the disposable test site is confirmed on 0.7.528D or later;
- 2 the installed-site installer and bootstrapper both refuse access;
- 3 the JSON-LD injection fixture no longer escapes its script element;
- 4 the editor account is refused by all three corrected system pages;
- 5 HSTS and the initial CSP appear in the live response headers;
- 6 this section is replaced with the deployment date and verification result;
- 7 the final PDF is rendered and visually inspected.